Advanced Algorithms 2025 Deadline: 1/12/2025, 11:59 pm

Graded Homework 2

Lecturer: Bernhard Haeupler, Johannes Lengler, Maximilian Probst

Note 1: *Solutions must be typeset in LaTeX and uploaded via moodle by 11:59 pm on December 1st 2025 (please use a separate pdf file for each problem). Late submissions will not be graded. If you would like to add a drawing in your solution, you can simply include a picture of a hand-drawn figure in your LaTeX. You can submit only one file for each problem and the drawing should be incorporated in the rest of your explanation.*

Note 2: *This is a theory course and any claim should be substantiated with a proof. When asked to give an algorithm, unless the problem statement states otherwise, the algorithm has to have polynomial running time. You are required to prove the claimed properties of any algorithm you present.*

Note 3: *Each student has to provide their own independent solution for the graded homework. You must complete the tasks without any external assistance from papers, books, online forums, or similar sources. The assignments are designed to be solvable using only the concepts covered in the lectures, and you are welcome to apply any theorems or lemmas from the lecture notes. The use of AI tools (such as ChatGPT) to generate ideas, provide feedback, formalize intuition, or otherwise help with the homework is not allowed. You are also not allowed to discuss the homework with other students; this includes both giving and receiving help.*

1 Server Management (20 points)

You are a developer of the popular social media application *ClickClock*. As the app grows in popularity, the backend team has problems coming up with a good method to distribute the user requests to servers, and they ask for your help. You come up with the following solution:

Let $\mathcal{U}$ be the universe of users and $\mathcal{S}$ be the collection of servers. We define $m \coloneqq |\mathcal{U}|$ and $n \coloneqq |\mathcal{S}|$ and assume that $m = \omega(n \log^2 n)$. Consider the unit circle $\mathbb{S}$ whose elements are parameterized by an angle in $[0, 2\pi)$. We map every user $u \in \mathcal{U}$ to a uniformly random point $\theta_u \in \mathbb{S}$. Similarly, we also map every server $s \in \mathcal{S}$ to a uniformly random point $\theta_s \in \mathbb{S}$. The random variables $\{\theta_s\}_{s\in\mathcal{S}}$ are fully independent and mutually independent of $\{\theta_u\}_{u\in\mathcal{U}}$. The random variables $\{\theta_u\}_{u\in\mathcal{U}}$ **do not** need to be independent. Now, we simply assign a user $u \in \mathcal{U}$ to

$$h(u) \coloneqq \underset{s\in\mathcal{S}}{argmin}\{\theta_s - \theta_u\}$$

which is the first server s whose point θ_s you encounter moving anti-clockwise in the circle starting from θ_u.

Your colleagues are still skeptical about the benefits of your scheme. So, you want to prove the following claims about your algorithm:

1. Show that in expectation, the number of users that are assigned to any server is m/n (6 Points).
2. If you now add a new server to $\mathcal{S}$, show that it suffices to remap at most m/n users in expectation (5 Points).

Even though this method works in expectation, it does not concentrate well. Fortunately, there is a simple modification (that we do not go into here) that fixes this problem. Thus, for the rest of the problem, you can assume for the sake of simplicity that $\{\theta_s\}_{s\in\mathcal{S}}$ are **deterministically equally spaced** points in $\mathbb{S}$.

3. Suppose that the random variables $\{\theta_u\}_{u\in\mathcal{U}}$ are fully independent. For any constant $\varepsilon > 0$, show that with high probability (i.e., with probability $1 - n^{-\Omega(1)}$), the number of users assigned to every server is at most $(1+\varepsilon)m/n$ (4 Points).

4. Your colleagues rightly point out that assuming full independence in (3) is impractical, as m can be very large. Show that the guarantees of (3) can still be retained by assuming that the random variables $\{\theta_u\}_{u\in\mathcal{U}}$ are k-wise independent for $k = \lfloor \log n \rfloor$ (5 Points).

2 K-means+++ (25 points)

You have seen the K-means++ algorithm in the lecture, but you are not satisfied with the approximation factor of $\mathcal{O}(\log k)$ it gives. Hence, you propose the K-means+++ algorithm, which is exactly the same as the K-means++ algorithm, but you run it for $100k$ rounds instead of k rounds. Show that this new algorithm achieves an approximation of 20 (with respect to the optimal solution using k centers) with probability $1 - e^{-\Omega(k)}$.
You can use the following lemma without proof.

Lemma 1 (Azuma-Hoeffding)**.** *Suppose that the indicator random variables $\{X_i\}_{1\le i\le t}$ satisfy $\mathbb{E}[X_i \mid \{X_j\}_{1\le j<i}] \ge p$ for all i. Then, $\mathbb{P}[\sum_{1\le i\le t} X_t \le (1-\delta)pt] \le e^{-\delta^2 p^2 t/2}$.*

Hint: Recall the concept of settled and unsettled clusters from the lecture.

3 High-Coverage Forest (25 points)

You are given an undirected graph $G = (V, E, w)$ with nonnegative edge weights $w(e) \ge 1$, vertex values $r(v) \ge 0$, a budget B, and a center limit $k \in [n]$. Your goal is to output a set $C \subseteq V$ of up to k centers along with a subset $T \subseteq E$ of edges of total weight $\sum_{e\in T} w(e) \le B$ not exceeding the budget, such that the total value of vertices $v \in V$ that can reach a center $c \in C$ in the subgraph (V, T) is maximized.

1. Show that reducing the budget by a t-factor for any integer $t \in \mathbb{N}_{\ge 1}$ asymptotically reduces the optimum value by at worst a t-factor, i.e. that

$$\mathrm{OPT}(G, B/t, k) = \Omega(\mathrm{OPT}(G, B, k)/t),$$

   where $\mathrm{OPT}(G, B', k)$ is the optimum value with budget B' (5 points).

2. Suppose that the graph G is a tree. Give a constant-approximate algorithm for this case (5 points).

3. Give a randomized algorithm for the problem that with high probability returns a valid solution T, C of value $\mathrm{OPT}(G, B, k)/O(\log n)$ (15 points).

For part 3, you may assume the results of parts 1 and 2, even if you did not complete those parts.

4 Approximate Flow from Shortest Paths (30 points)

Let $G = (V, E, u)$ be a directed graph with capacities $u(e) \ge 1$.
Recall that the dual of the maximum s-t -flow problem is an assignment of lengths $\ell(e) \ge 0$ to edges, such that the distance $\mathrm{dist}_\ell(s, t)$ from s to t in the graph G with edge lengths ℓ is at least

1, and the total weighted length $\langle \ell, u \rangle = \sum_e \ell(e) \cdot u(e)$ of edges is minimized. By LP duality, the optimum value β of the dual is equal to the optimum value of the primal, i.e. the maximum value $\text{val}(f)$ of any capacity-respecting flow f.

Consider the following algorithm, parameterized by $0 < \epsilon, \delta < \frac{1}{2}$ chosen later: let $f_0 = 0$ be an empty flow, and initialize the length of each edge to $\ell_0(e) = \delta$. Then, repeat the following for $i = 1, 2, 3, \ldots$:

- If $\text{dist}_{\ell_{i-1}}(s, t) \geq 1$, break.
- Otherwise, let P_i be a shortest path from s to t in G with edge lengths ℓ_{i-1}, and let $u_i^{\min}$ be the minimum capacity $u(e)$ of an edge $e \in P$. Then,
  - Update the flow by pushing $\epsilon \cdot u_i^{\min}$ flow along P_i, i.e. by letting $f_i := f_{i-1} + \epsilon u_i^{\min} P_i$.
  - Update the length of each edge to $\ell_i(e) := \ell_{i-1}(e) \cdot \left(1 + \frac{f_i(e) - f_{i-1}(e)}{u(e)}\right)$.

Suppose that the break condition was satisfied in iteration $i = k + 1$. Let $f := f_k$ and $\ell := \ell_k$.

1. Show that the algorithm performs at most $k = O\left(m \cdot \frac{\ln(1/\delta)}{\epsilon}\right)$ iterations (6 points).
2. Show that f has congestion (i.e. $\max_e \frac{f(e)}{u(e)}$) at most $\frac{\ln \frac{1}{\delta}}{1-\epsilon} + O(1)$ (6 points).
3. Let $d_i := \text{dist}_{\ell_i}(s, t)$. Suppose that $d_i > \delta n$. Argue that $\frac{1}{d_i - \delta n}(\ell_i - \ell_0)$ is a valid solution to the dual, and thus in particular $\langle \ell_i, u \rangle - \langle \ell_0, u \rangle \geq (d_i - \delta n)\beta$ (5 points).
4. Show that $\text{val}(f) \geq \beta \cdot \left(\ln(\frac{1}{\delta n}) - O(1)\right)$ (9 points).

   Hint: Show that $d_i \leq \delta n \cdot \exp\left(\frac{\text{val}(f_i)}{\beta}\right)$ for $0 \leq i \leq k$. To do this, bound the increase to $\langle \ell_i, u \rangle$ in iteration i in terms of d_{i-1}, ϵ, and $\text{val}(f_i) - \text{val}(f_{i-1})$, then combine with part 3.
5. Selecting δ appropriately, conclude that $O\left(\frac{m \ln n}{\epsilon^2}\right)$ shortest path computations are sufficient to obtain an $(1 - O(\epsilon))$-approximate maximum s-t -flow (4 points).

For each part, you may assume the results of earlier parts, even if you did not complete those parts.